Curtin University – Department of Computing

Assignment Cover Sheet / Declaration of Originality

Complete this form if/as directed by your unit coordinator, lecturer or the assignment specification.

Last name:	Nguyen	Student ID:	23857030
Other name(s):	Thanh Dat		
Unit name:	Data Structures and Algorithms	Unit ID:	COMP5008
Lecturer / unit coordinator:	Ying Li	Tutor:	Ying Li
Date of submission:	30/10/2026	Which assignment?	(Leave blank if the unit has only one assignment.)

I declare that:

- The above information is complete and accurate.
- The work I am submitting is *entirely my own*, except where clearly indicated otherwise and correctly referenced.
- I have taken (and will continue to take) all reasonable steps to ensure my work is *not accessible* to any other students who may gain unfair advantage from it.
- I have *not previously submitted* this work for any other unit, whether at Curtin University or elsewhere, or for prior attempts at this unit, except where clearly indicated otherwise.

I understand that:

- Plagiarism and collusion are dishonest, and unfair to all other students.
- Detection of plagiarism and collusion may be done manually or by using tools (such as Turnitin).
- If I plagiarise or collude, I risk failing the unit with a grade of ANN ("Result Annulled due to Academic Misconduct"), which will remain permanently on my academic record. I also risk termination from my course and other penalties.
- Even with correct referencing, my submission will only be marked according to what I have done myself, specifically for this assessment. I cannot re-use the work of others, or my own previously submitted work, in order to fulfil the assessment requirements.
- It is my responsibility to ensure that my submission is complete, correct, and not corrupted.

Signature: Thanh Dat Nguyen Date of signature: October 29, 2026

(By submitting this form, you indicate that you agree with all the above text.)

Biogas Feedstock Collection Network

COMP5008 FINAL ASSIGNMENT / STUDENT ID 23857030

1. System Overview

The system models the collection of potential organic feedstock from a range of farms across Western Australia and transport of that feedstock to a proposed biogas hub. It brings together farm lookup, route calculation, request prioritization, and historical data analysis in a single application. The model shows the data structures; the available feedstock and transport times are all illustrative scenarios and not actual production or road routing data.

Source farms	Test vertices	Test edges	Passing tests
125	12	18	239

Module	Responsibility	Information exchanged
Graph	Adjacency-list network; BFS, DFS, and manual Dijkstra.	Farm/hub IDs -> minimum T and path.
Hash Table	Manual modulo hashing, probing, insert/search/delete.	FarmID -> complete FarmRecord.
Heap	Calculated P=F/(T+1); manual array Max Heap.	Request + farm + route -> collection order.
Sorting	Manual Merge/Quick Sort; persistent request history.	Snapshots/status -> ascending-T records.

For C001 the lookup finds farm doj25; Dijkstra returns 10 minutes; educational F=11 tonnes gives P=11/(10+1)=1. The heap schedules the request, and the archive takes a snapshot. Dispatch then rechecks the same hash table. The historical records are then sorted by T for analysis, separate from the heap priority.

Reproduction code

```
python interactive_menu.py
python interactive_menu.py --integrated-demo
python -m unittest discover -s . -v
```

2. Dataset and Data Preparation

2.1. Region, collection provenance and preparation

The selected region is Western Australia, Australia. The provided farm_dataset_red_meat_WA.csv contains 125 farm records and 20 fields, all marked as WA and Australia. This is a selected snapshot from the Farm Transparency Project's public map/facility data [1] and is not in any way a claimed census of all the farms in WA. The dataset includes the source name, coordinates, animal species, classifications, and addresses.

The farm data was collected on 19th September 2026 from the Farm Transparency Project website via the interactive map (WA as the region of interest, farm records pulled). This uses the CSV snapshot rather than a fresh scrape. Each record has a traceable Map URL and Profile URL. Preparation checks were carried out to confirm ID/URL agreement for all 125 rows. We did a spot check of 5 records on 19th September 2026. data/source_verification.csv contains the fields that were checked and the limitations.

Input	Origin and handling
ID, Name, Species, Lat, Lng	Public-source fields. FarmIDs remain strings; names are not unique keys.
Street Num, Street, Suburb, State, Postcode, Country	Required address fields. Source blanks are retained; none are inferred.
Nine additional source fields	Preserved in each complete immutable FarmRecord, including source URLs and status qualifiers.
Hub and network edges	Student-selected modelling locations, connections, road factor, and speed
EstimatedFeedstock / priority	Educational F supplied in requests; P calculated by the program, never a source production claim.

Exact website IDs and data integrity

Three uploaded ID cells had spreadsheet scientific-notation corruption. The exact IDs were restored from agreeing source URL fields and checked against public listings; no farm was renumbered. All other dataset field values and row order are preserved.

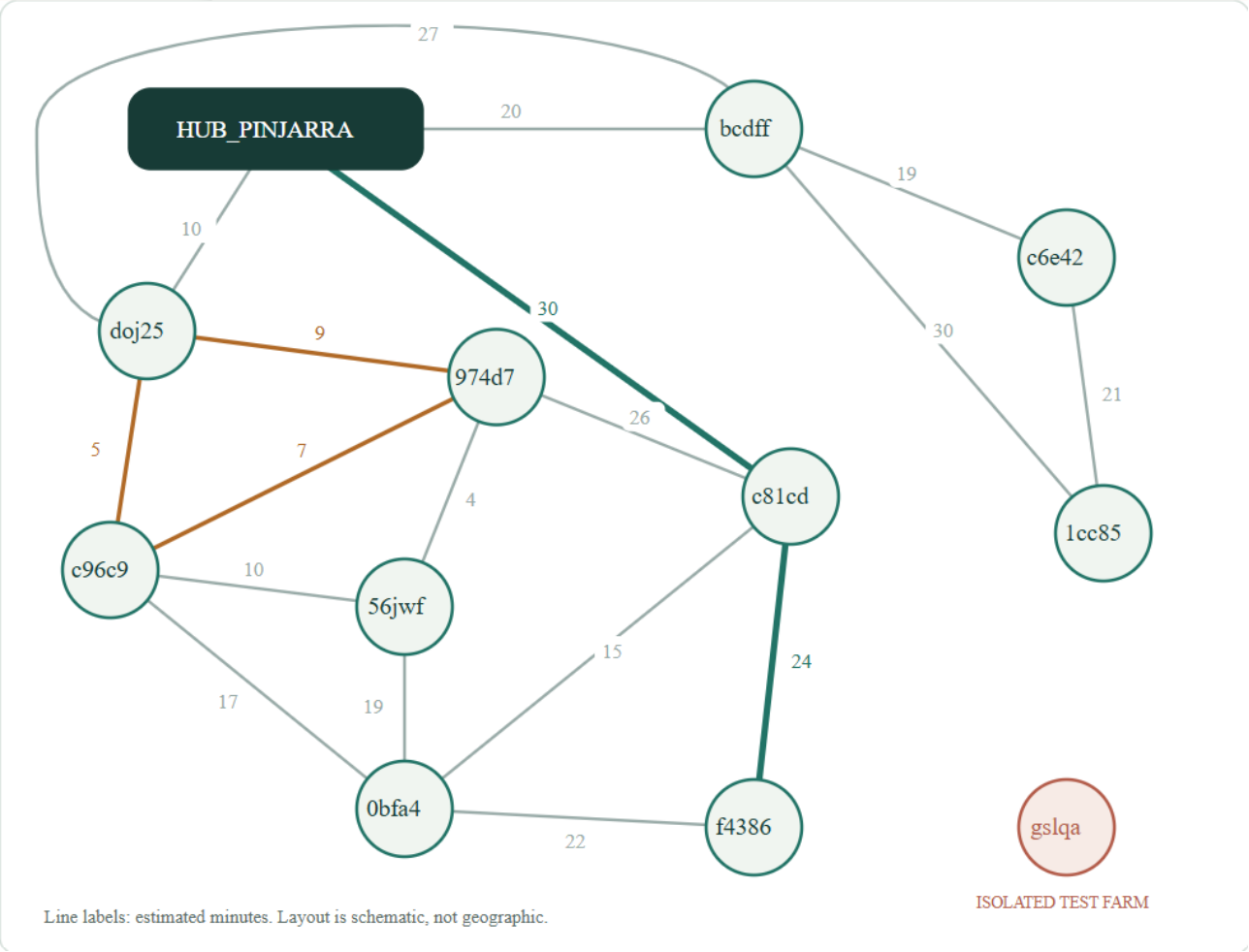
Uploaded cell	Exact website ID	Farm
6.50E+31	65e30	Paradise Beef Cattle Feedlot
6.00E+64	60e63	Lake Preston Cattle Feedlot
8.00E+06	80e05	Twyata Pool Feedlot

data/farm_dataset_red_meat_WA.csv preserves the uploaded bytes; id_corrections.csv is the change log. ID, name, and species: contain values and are not missing. Latitude and longitude: contain values and are finite numbers. Blank street numbers/roads/suburbs/postcodes (23/1/3/3) remain blank. The source unconfirmed category for p2vf9 is retained. Keep FarmID columns as text when importing CSVs.

2.2. Hub and Transportation Network

The proposed HUB_PINJARRA uses approximate town coordinates (-32.6285, 115.8750). It's situated in the vicinity of the selected West Pinjarra/Nirimba/Nambeelup/Waroona farm cluster, so a fairly compact teaching network is feasible. There is no existing biogas facility, no approved site, and no identified optimal hub. The hub is stored separately from the public farm records in data/hub.csv.

For each selected connection, calculate the haversine great-circle distance d (Earth radius = 6,371 km). Road distance is assumed to be 1.30d and average speed 60 km/h. Estimated minutes are max(1, ceil(60 x 1.30d/60)). I give positive weights in both directions. This is a documented approximation which ignores actual roads, traffic, loading, and vehicle limitations.



Test network: 11 real farms plus a hub, 18 explicit undirected connections, a cycle and a deliberately isolated gslqa. Isolation is a validation scenario, not evidence of physical inaccessibility. The layout above is, of course, schematic, with the labels all modelled in minutes.

Full mode = all 125 farms plus the hub (126 vertices, 254 edges). The rest of the locations are connected via minimum great-circle links, with 3 nearest-neighbour alternatives also added (duplicates removed). gslqa is a singleton. ID tie-breaking is used so that the generated CSVs are reproducible. full_edges.csv and test_edges.csv contain the respective estimates.

3. Data-Structure Justification

3.1. Graph: adjacency lists and route algorithms

A weighted undirected graph represents the shared transport links between the farms and the hub. The adjacency lists contain only the relevant links and are perfectly suited to this very sparse teaching network. Each logical edge is represented twice. Dynamic insertion checks both endpoints and a positive finite weight before either adjacency list is altered. Duplicate IDs/edges and self-loops are all rejected. Two farms with the same name and the same coordinates but different FarmIDs are considered to be distinct.

Algorithm	Why it fits this system
BFS	A custom linked queue. BFS visits reachable locations in FIFO order, grouping locations by minimum number of connections from the source, not representing total travel time.
DFS	An iterative colour/parent traversal visits all components and reconstructs a cycle from an active ancestor. The undirected reverse parent edge is ignored.
Dijkstra	Positive weights permit minimum-time routes [2]. A manual linear scan selects the next unsettled location; predecessor links reconstruct the path.

Isolated and disconnected locations are still graph vertices. An invalid ID gives rise to a GraphError.

An unreachable route returns infinity and an empty path (the UI makes this perfectly obvious). A route from a node to itself is zero. We do iterative traversal to avoid hitting Python's recursion limit. We make absolutely no use of any graph or shortest path library.

3.2. Hash Table: open addressing with linear probing

Farm lookup uses one NumPy array of EMPTY, DELETED, and FarmRecord slots. Open addressing eliminates the need for separate node allocation and makes the collision/probe sequences trivial to show. The core index makes no use of any built-in dictionary/map, set, or built-in hash(). A polynomial modulo hash is used, which accumulates h=(31h+ord(c)) mod m for each of the 5 ID characters. Probes then visit (h+i) mod m until a matching ID is found or a never-used slot is encountered.

Initial capacity is the prime 11. Growth picks a prime which is at least twice the old capacity: 11 -> 23 -> 47 -> 97 -> 197. If the projected active load is greater than 0.70, or there is no usable slot, the table grows prior to insertion of a new record. Otherwise, if the projected occupied load including tombstones exceeds 0.85, the table rehashes at the same capacity to eliminate tombstones. When a tombstone is reused, the occupied count is not incremented. Linear step 1 visits every slot for any capacity.

Duplicate FarmIDs are rejected, not silently replaced. Deletion leaves a tombstone so later keys in the cluster stay searchable. Search/delete of a missing valid ID returns None with readable UI messages. No automatic shrinking is used; cleanup and reusable tombstones reduce deletion/reinsertion thrashing.

3.3. Heap and Sorting Choices

The collection scheduler uses an array-based Max Heap (larger priority = earlier collection). A validated request contains the estimated feedstock F, the shortest transport time T and its path calculated using Dijkstra, and a snapshot of the farm record. P is calculated as F/(T+1 minute) so that increasing F increases the priority and increasing T decreases the priority. T=0 gives a safe P=F. For exact priority ties, we use the FIFO arrival order with full precision comparisons.

Insertion appends and percolates up via the parent (i-1)//2. Extraction involves replacing the root with the final entry and then percolating down towards the higher of the two children (2i+1 or 2i+2). Peek reads the root value without any modification. A successful insertion or extraction will always print out the indexed array and the swap trace. Empty and singleton heaps are dealt with explicitly.

3.4. Bottom-up Merge Sort

The algorithm sorts T in ascending order. Bottom-up merge sort combines runs of width 1, 2, 4, etc., and alternates source and output buffers. It avoids the depth limitations of recursion and gives guaranteed O(n log n) work. On equality, the left record is taken, which preserves input order, as is the case with the normal stable bottom-up method [3]. There is no shortcut for the already sorted run in this implementation. Merge is the archive default.

3.5. Median-of-three, three-way Quick Sort

Choose the median key from the first, middle, and last positions. Partition the list into keys less than, equal to, and greater than that pivot. Median-of-three is a standard pivot heuristic [4], and three-way partitioning is used to group the equal keys together [5]. This is a manual implementation which combines those two ideas without resorting to a library sort. The smaller side is processed first, and the larger side is pushed onto an explicit stack.

On the initial input (first, middle, and last sampled keys all different), the first pivot cannot be either the minimum or the maximum key. If the sampled keys are not all different, then the pivot could still be an extreme value. Later partitions can still be unbalanced, so the quadratic worst case remains. Quicksort is unstable (the farm events and the rounded minute times make the equal key behaviour of the algorithm relevant). The in-place API can reduce the amount of auxiliary storage required; the benchmark wrapper will, of course, copy the input to give both algorithms the same non-mutation contract.

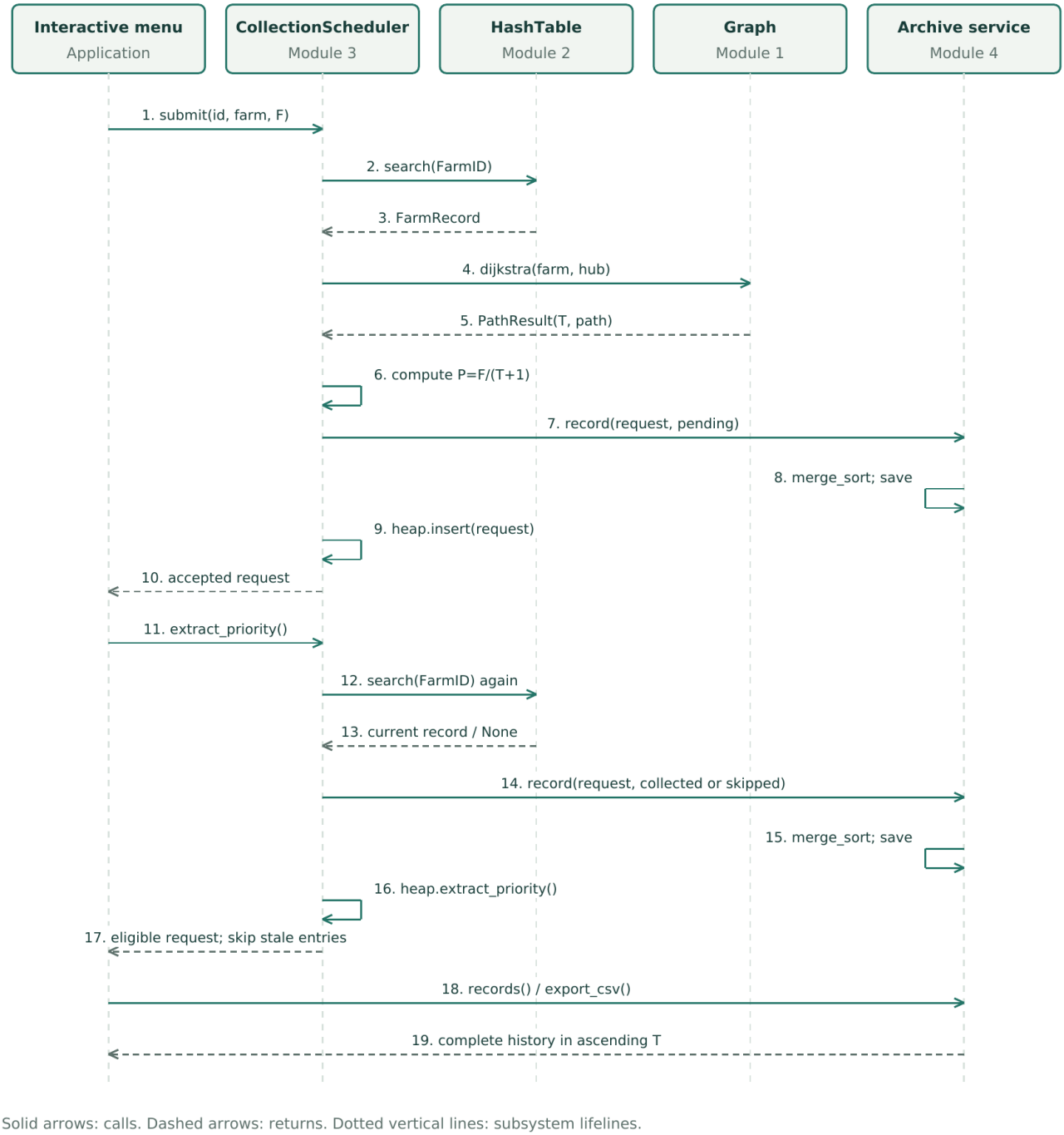
Input (identity: T)	Stable Merge output	Quick output
A:2, B:2, C:1	C:1, A:2, B:2	C:1, B:2, A:2

Both outputs are correctly sorted; only Merge preserves A before B. Stability is relative to the input of that call and does not restore an order changed by a previous unstable sort. Choosing Merge for history prioritizes stable ties and predictable bounds; Quick is retained for comparison and its in-place option.

4. Module Integration

4.1. UML Sequence Diagram

The lifelines represent application/module boundaries. Module 3 contains its MaxHeap; the Module 4 archive lifeline is the combination of SessionRecorder and CollectionDatabase. The calls shown below are those corresponding to the implemented accepted-request/dispatch flow.



Solid arrows: calls. Dashed arrows: returns. Dotted vertical lines: subsystem lifelines.

The collected/skipped branch records the current farm-check outcome before extraction. Stale entries are skipped until an eligible request is found or the heap becomes empty. Editable UML is supplied as docs/module_integration_uml.puml.

4.2. Data Flow and Shared State

Caller -> receiver	Data passed / result.	Use or validation
Menu -> scheduler	RequestID, FarmID, F	Validate syntax, finite positive F, duplicate request/pending farm.
Scheduler -> hash	FarmID -> complete FarmRecord	Reject absent/deleted records; dispatch reuses the same lookup.
Scheduler -> graph	FarmID, hub ID -> T and path	Check location and coordinates; reject unreachable routes.
Scheduler -> MaxHeap	ScheduledRequest with computed P	Percolation maintains priority/FIFO order; arrays are printed.
Scheduler -> archive	Request snapshot + status/detail	Preserve all accepted requests, including skipped/cancelled history.
Archive -> sorter	CollectionRecord list, key=T	Manual sorting calculates positions before atomic SQLite storage.
Benchmark -> modules 1-3	Hash records + cached Dijkstra paths	Reuse validated ScheduledRequest to calculate P for historic event scenarios.

One live system, not four isolated runs

The root menu has one graph, one shared HashTable, and one archive. Module 2 deletion is immediately visible to a queued Module 3 request. Hash reload rebinds that existing scheduler to the new index. A change in graph vertex/edge count will block the old priorities until they are reset. Network reload will cancel the pending history and clear the scheduling session. Both reset and application exit will retain the cancelled records.

Module 3 takes an archive recorder in its constructor (so to speak) and doesn't import a specific database implementation. That keeps the queue logic modular, and Module 4 gets the actual ScheduledRequest snapshot. Session-prefixed RecordIDs mean a RequestID can be reused in a subsequent session without risk of overwriting the previous history. SQLite stores the explicit sort positions and reads them in position order; there is no SQL ORDER BY on the transport key.

Executed end-to-end demonstration

integration_demo.py and root option 13 use the real classes in a single fresh session. The normal scenario archives and collects all 10 requests. The deletion scenario archives 10, collects 9, and retains C001 as skipped following the deletion of doj25. Instance identity checks and call spy tests confirm the shared objects and the actual Dijkstra/hash/manual sort calls. The saved CSV and SQLite snapshots all round-trip perfectly.

Invalid farm/request data causes a typed error prior to queue insertion. If an archive write fails, then submit/extract will report an error prior to mutating the heap. Normal menus will never rewrite the source files. Output directories are configurable, and input files resolve relative to the program rather than the working directory.

5. Algorithm Complexity

5.1. Graph and Hash Complexity

V is the number of graph vertices, E is the number of undirected edges, Vr/Er are the source reachable components. m is hash capacity, q is pending requests, n is sorting records, r is archived requests. The tables are for constant-time numeric comparisons/key access and 5-character ID hashing. Space is auxiliary unless otherwise stated. Graph lookup uses a custom, expected O(1) modulo-hash index, and Module 2's core hash index is still manual.

Algorithm	Best case	Average / expected	Worst case	Aux. space
BFS	O(V) with isolated source	O(V+Vr+Er); O(V+E) bound	O(V(V + E))	O(V)
DFS, all components	O(V+E)	O(V+E)	O(V(V + E))	O(V)
Dijkstra, linear scan	O(V), source= destination	O(V^2+E) bound; depends on settlements	O(V^3 + VE)	O(V)
Hash search/delete	O(1)	Expected O(1), bounded load and dispersed keys	O(m)	O(p), worst O(m)
Hash insert, no resize	O(1)	Expected O(1), same assumptions	O(m)	O(p), worst O(m)
Hash insert incl. resize	O(1)	Expected amortised O(1) over growth	O(m^2), severe rehash clustering	O(m) at resize

Traversal and shortest-route factors

BFS scans reachable adjacency records and then scans all vertex IDs to list unreachable locations, so an isolated-source call still costs O(V). DFS deliberately visits every component and processes each adjacency entry; reconstructing one cycle costs at most O(V). For Dijkstra, each settled vertex requires a full V scan, followed by edge relaxation. With k settlements and Es scanned adjacency entries, the expected work is O(V + kV + Es), assuming expected O(1) graph-map operations. Early stopping can reduce observed work, but this implementation has no binary-heap Dijkstra bound.

Hash assumptions and the cost of growth

With the default thresholds, the hash table grows if an insertion would push the active load factor above 0.70 or no usable slot remains. If growth isn't needed but the projected occupied load, including tombstones, would exceed 0.85, the table rehashes at the same capacity to remove tombstones. With well-distributed keys, these thresholds keep probe sequences short and support expected O(1) search and deletion. In the worst case, however, primary clustering and tombstones can cause an operation to examine O(m) slots, where m is the table capacity.

During growth, a larger prime capacity is chosen, and the live records are reinserted. With well-distributed keys, the reinsertion takes expected O(m) time, and geometric growth gives us expected amortized O(1) insertion. In the case of very severe collisions, reinsertion of the records can take as long as $O(m^2)$. There is also the cost of prime selection and array allocation, and both the old and new arrays may be in existence at the time of resizing.

The _probe() method records each slot it looks at for the purpose of tracing. If an operation looks at p slots, then its trace will require O(p) auxiliary space. This is O(1) expected space for short probe sequences and O(m) in the worst case.

5.2. Heap and Sorting Complexity

Operation	Best case	Average / expected bound	Worst case	Aux. space
Heap inserts: percolation.	O(1)	O(log q); distribution dependent	O(log q)	O(log q) saved swaps
Heap extract: percolation	O(1)	O(log q); distribution dependent	O(log q)	O(log q) saved swaps
Heap peek	O(1)	O(1)	O(1)	O(1)
Bottom-up Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Three-way Quick core	O(n), all equal	Usual O(n log n); no random-input guarantee assumed	O(n^2)	O(log n) stack

Heap work vs Complete system operations

The heap has height O(log q), limiting comparisons/swaps in each percolation. No probability model is assumed for request priorities, so O(log q) is an average-case upper bound rather than a claimed tight expected cost. NumPy object-array resizing may occasionally copy all O(q) references; allocation is amortized. The required full-array trace itself adds O(q) formatting/output for every mutation.

Request preparation also includes expected O(1) farm lookup, expected O(V^2+E) Dijkstra, and the saved path. Dispatching past k stale requests performs up to k+1 heap extractions and current-record checks. With persistence enabled, each archive update manually re-sorts r records in O(r log r), serializes snapshots, and performs SQLite I/O. Those costs are additional to heap percolation and are excluded from the sorting timing experiment.

Sorting bounds, stability and actual space

Merge performs all doubling-width passes, writing n entries each pass, even when input is ordered. Its O(n log n) bound therefore applies to best, average, and worst cases. Quick's equal-region finish in one partition gives O(n) for all equal keys. Balanced distinct-key partitions give O(n log n); highly unbalanced partitions can still give O(n^2). Median-of-three helps pivot selection but is not a worst-case safeguard.

Processing the smaller Quick side first bounds the deferred stack by O(log n), including bad-pivot cases; it avoids recursive overflow without removing quadratic time. Benchmark wrappers copy their inputs, so both submitted non-mutating sort APIs use O(n) total extra storage. Merge additionally uses an O(n) merge buffer; quick_sort_in_place avoids the output copy and uses O(log n) stack space. Copies contain references rather than deep-copied farm snapshots. Memory was not measured.

6. Experimental Results

6.1. Graph Test Results

The executable graph driver and 62 graph tests pass. Eight seeded six-vertex fixtures are checked against an independent exhaustive simple-path oracle for every ordered pair: 288 shortest-path comparisons. Returned routes are also checked by summing their actual edge weights. Traversal of a 1,500-vertex chain verifies iterative operation.

Graph specification	Test network	Full network
Vertices / undirected edges	12 / 18	126 / 254
Adjacency records	36	508
Connected components	2	2
Cycle exists	Yes	Yes
Isolated farm	gslqa	gslqa
Weights	Finite and positive	Finite and positive

BFS from HUB_PINJARRA: levels by edge count

Level	Reachable IDs
0	HUB_PINJARRA
1	doj25, bcdff, c81cd
2	c96c9, 974d7, c6e42, 1cc85, 0bfa4, f4386
3	56jwf

gslqa is reported unreachable. BFS groups by the number of connections, not the sum of transportation minutes. An isolated source reaches itself at level 0.

DFS cycle witness and Dijkstra routes

DFS scans all components and detects doj25 -> c96c9 -> 974d7 -> doj25. Parent tracking avoids mistaking a normal reverse undirected edge for a cycle. The isolated location is still included in component traversal.

Query from hub	Shortest path	Time
f4386 (test)	HUB_PINJARRA -> c81cd -> f4386	54 min = 30 + 24
doj25 (test)	HUB_PINJARRA -> doj25	10 min
gslqa	No reachable path	Unreachable
HUB_PINJARRA	Source equals destination	0 min
NOT_A_LOCATION	Readable invalid-location error	No route requested

Invalid weights, duplicate vertices/edges, disconnected cycles, equal-cost ties, and routes with more hops but less total time are covered. Full adjacency/traversal logs are supplied as test_network.txt and full_network.txt.

6.2. Hash Table Results

All 125 complete records are inserted and are searchable. The 52 hash tests cover 1,000 mixed operations against an independent oracle, duplicate/invalid IDs, missing records, full field retention, and collision/deletion. Blank source addresses are valid; blank names and invalid/non-finite coordinates are rejected.

Explicit collision: fresh 11-slot table

Operation	Intermediate state	Result
Insert 3d8af	home=5; slot 5 EMPTY	Stored at 5
Insert 71a14	home=5; 5 occupied by 3d8af -> 6 EMPTY	Collision resolved at 6
Delete 3d8af	slot 5 becomes DELETED	Cluster is retained
Search 71a14	DELETED -> 6 contains 71a14	Found after deletion.

Bulk import: load factor after every ten insertions

Inserted	Active	Capacity	Load factor
10	10	23	0.4348
20	20	47	0.4255
30	30	47	0.6383
40	40	97	0.4124
50	50	97	0.5155
60	60	97	0.6186
70	70	197	0.3553
80	80	197	0.4061
90	90	197	0.4569
100	100	197	0.5076
110	110	197	0.5584
120	120	197	0.6091

Growth prior to insertions: 8, 17, 33, 68; capacity changes: 11 -> 23 -> 47 -> 97 -> 197. Load drops following a resize and then increases with further insertions. Final alpha is 125/197 = 0.6345. There are 277 insertion-operation probes (2.216 per inserted record), 52 inserts with placement collisions, and 167 additional rehash probes. These are the numbers for this dataset and do not constitute proof of the expected complexity.

Successful and unsuccessful search and deletion logs, the pre- and post-states, and all the checkpoints are in hash_demo.txt, hash_validation.json, and hash_load_factors.csv. Missing key deletion is safe in that deleting a record doesn't make a later cluster member unsearchable.

6.3. Heap Scheduling Results

The main test run takes ten requests, extracts ten, and then prints out every resulting heap array and percolation swap. F is educational tonnes; T is the actual output from Dijkstra on the submitted test graph. None of those priorities are entered by hand.

Request	FarmID	F tonnes	T min	P=F/(T+1)
C001	doj25	11	10	1.000000
C002	c96c9	8	15	0.500000
C003	974d7	15	19	0.750000
C004	bcdff	21	20	1.000000
C005	56jwf	6	23	0.250000
C006	c81cd	15.5	30	0.500000
C007	0bfa4	16.5	32	0.500000
C008	c6e42	24	39	0.600000
C009	1cc85	25.5	50	0.500000
C010	f4386	27.5	54	0.500000

> **Resulting collection order**
> C001 -> C004 -> C003 -> C008 -> C002 -> C006 -> C007 -> C009 -> C010 -> C005

C001 and C004 tie at P=1, so C001 arrives first and is collected first. The five P=0.5 requests also retain FIFO order. Appendix A lists all 21 primary states, including the final empty extraction attempt. Full indexed priorities/swaps are in heap_traces.txt.

Edge case	Observed/validated response
Unreachable gslqa	Reject with a clear no-valid-route message.
T=0	Separate real-farm-as-hub fixture: Dijkstra gives 0; F=5 yields P=5. No zero edge weight is added.
Invalid/missing/deleted farm	Reject before insertion; deleted queued records are skipped on extraction.
Invalid F / duplicates	Reject non-positive/non-finite/non-numeric F, repeated RequestID, or second pending.
Empty heap / changed state	Return None safely; stale peek reports without removal; graph mutation requires reset.

All 63 heap/scheduler tests pass, including a seeded 1,000-operation independent priority oracle. A farm may receive a new request after collection; replayed accepted IDs remain duplicates within a session. The full graph recalculates times/priorities.

6.4. Sorting Experiment Design

The sorting generator loads the same 125 source farms into the manual Hash Table and calls actual Dijkstra for each farm in full mode. It caches 124 reachable T/path results; gslqa is explicitly excluded. It generates 1,000 unique historic request events at those real farms, with educational F of 2-30 tonnes and Module 3-calculated priority. T is copied from the route cache, not generated as an unrelated random number.

Repeated FarmIDs represent separate historic events, not invented farms or concurrent pending requests. Prefixes of the same base produce sizes 100 and 500. At each size, the three conditions contain identical records. Seed 23857030 fixes the base; the condition seed is seed + size*10 + condition index (0/1/2).

n	Condition	Displaced IDs	Adjacent descents	Distinct T
100	random	99	51	67
100	nearly sorted	10	10	67
100	reversed	100	66	67
500	random	500	247	113
500	nearly sorted	50	48	113
500	reversed	500	112	113
1000	random	999	511	113
1000	nearly sorted	100	94	113
1000	reversed	1000	112	113

Our own stable Merge Sort produces the ascending baseline. Random uses a seeded shuffle; reversed swaps endpoints inward. Nearly sorted swaps disjoint sampled positions, displacing exactly 10/50/100 record identities: 10% at every required size. Displacement includes moves among equal-T records and is verified relative to that baseline. Input hashes and exact CSVs are supplied.

Measurement and fairness

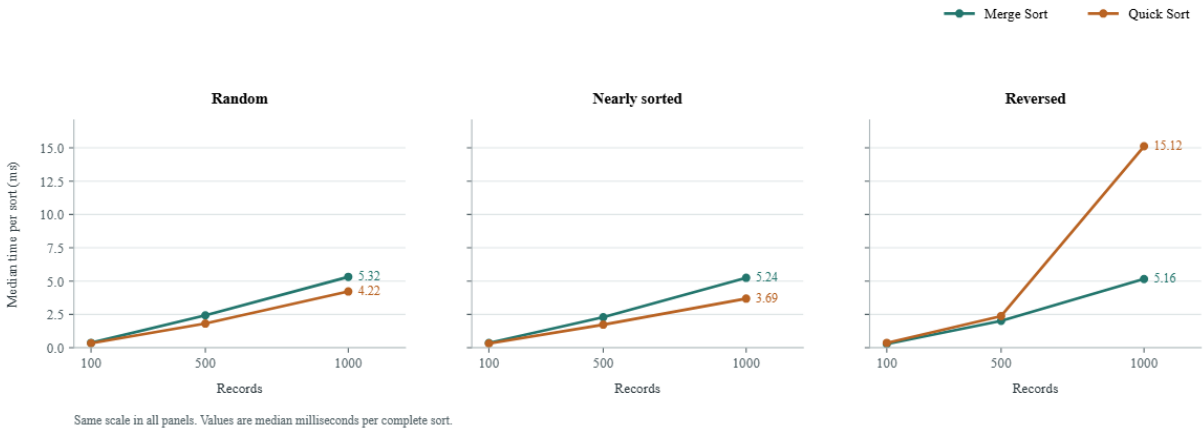
For each algorithm/case, do one untimed warm-up/correctness check, then time it 7 repeats x 25 complete sorts. Report median batch time / 25 in ms. Result copies are included; dataset/route generation, metrics, validation, printing, and CSV/SQLite I/O are all excluded. Each call is given the same unaltered input; the middle condition reverses the order of the algorithm. GC is disabled during timing via timeit [6]. The median is the value we report from this experiment; the minima/maxima and all 126 raw batch-normalized samples are available for host-load assessment should you wish to look at them.

Seeded data is reproducible; elapsed time varies with hardware and load. Algorithm order is not fully randomized, memory was not measured, and no confidence interval or statistical-significance claim is made.

6.5. Sorting Timing Results

The table reports median milliseconds per sort. Quick / Merge below 1 means Quick was faster in that measured case. Values are experimental observations, not Big-O bounds.

n	Condition	Merge	Quick	Quick/Merge
100	random	0.365796	0.337636	0.923
100	nearly sorted	0.360740	0.321604	0.892
100	reversed	0.279520	0.363648	1.301
500	random	2.431920	1.814360	0.746
500	nearly sorted	2.287624	1.729504	0.756
500	reversed	2.017116	2.376216	1.178
1000	random	5.315344	4.220088	0.794
1000	nearly sorted	5.244908	3.687876	0.703
1000	reversed	5.156768	15.116008	2.931



Both algorithms produced nine correctly sorted outputs, preserving the complete record multiset; all nine Merge outputs preserved equal-T input order. There are 18 sorted CSVs in sample_output/sorting/. Merge was faster in three cases and Quick in six on this run. Raw ranges should accompany small n=100 differences, where some batch ranges overlap. Another host/run can produce a different ranking.

6.6. Sorting Performance Analysis

Dataset growth: observed ratios versus theory

Condition	Merge	Quick
random	14.53×	12.50×
nearly sorted	14.54×	11.47×
reversed	18.45×	41.57×

Both algorithms took longer at each required size increase. For comparison, n*log2(n) grows about 15.0x from 100 to 1,000. Fixed overhead, allocation, tie frequency, partition balance, and host load affect measured ratios. Three sizes do not establish an asymptotic law. Section 5 gives theoretical bounds independently of these timings.

Input condition and the chosen pivot

Merge was fastest on reversed input at every size. Ordered runs can exhaust one side earlier, reducing comparisons while retaining all width passes. At n=1,000, random and reversed Merge use 8,671 and

5,710 comparisons respectively, but both write 10,000 merge-buffer entries. This implementation does not detect and skip sorted runs.

Quick groups equal keys: n=1000 has only 113 distinct T values. Nearly sorted input was faster than random for Quick at n=100, n=500 and n=1000. The number of displaced records in themselves doesn't determine the runtime; their positions and the balance of the pivot later on do. For an ordered range, if the first, middle, and last sampled keys are all distinct, then median-of-three will pick a first pivot which is neither the minimum nor the maximum key. If the sampled keys contain duplicates, then the pivot can still be an extreme value. Subsequent partitions can also become unbalanced as a result of the swaps.

n=1000 input	Quick comparisons	Partitions	Max deferred ranges
random	11201	113	4
nearly sorted	11774	113	4
reversed	18986	113	5

Reversed input required 18,986 Quick comparisons versus 11,201 for random, and was Quick's slowest condition. These cases show arrangement sensitivity, not a measured quadratic family. Minute rounding, repeated farm events, comparison/key-access costs, and duplicate distribution also influence observed time.

Reflection on algorithm choice

The archive defaults to stable Merge Sort for predictable worst-case work and preservation of equal-T input order. Quick offers an in-place option and measured wins in some cases; those wins do not give a universal guarantee. Its instability and O(n^2) worst-case risk remain. The relevant space distinction is the in-place core versus the O(n) copying benchmark APIs, not a claim of constant-space Quick Sort.

6.7. Integration and Verification

239 tests pass across all four modules and the complete system. The eight final integration tests verify shared object instances, actual algorithm calls, changed farm records, persistence and CLI/menu behaviour.

Area	Passing tests	Evidence
Graph	62	288 independent route-oracle comparisons, cycles/disconnection and iterative traversal.
Hash	52	Complete records, probes, tombstones, resizing, and mixed-operation oracle.
Heap	63	Priority, FIFO ties, 1,000 mixed oracle operations and cross-menu invalid/stale cases.
Sorting	54	Both sorts, stability, all nine conditions, dataset integration, and SQLite/CSV round trips.
End-to-end	8	Normal 10-collected flow and deleted-farm 9-collected/1-skipped history.

How to reproduce from the extracted project folder

```
python interactive_menu.py
python interactive_menu.py --integrated-demo
python integration_demo.py --delete-farm doj25
python interactive_menu.py --module graph --validate
python interactive_menu.py --module hash --validate
python interactive_menu.py --module heap --validate
python interactive_menu.py --module sort --validate
python -m module4_sorting.benchmark
python -m unittest discover -s . -v
```

The root menu has the graph/hash/archive state in common. Options 10, 11, and 12 open the lookup, scheduler, and sorted history, respectively. Option 13 runs a fresh complete demo without clearing the active session.

--output-dir selects a runtime output location. The runtime/tests make use of NumPy and the Python standard library. The independent demos and benchmark artifacts are all clearly named, and the submitted source data remains read-only.

7. Reflection and Design Trade-Offs

This project has really helped me to get a better understanding of how various data structures and algorithms can be combined to solve a real-world problem. One key takeaway was that when you're selecting an algorithm, you need to look at it in terms of its theoretical complexity as well as the characteristics of the data, the requirements of the implementation, and the way it will interact with the other parts of the system.

For Module 1, I went with an adjacency list graph as the transport network has quite a low degree of connectivity (in comparison to the total number of possible connections). BFS and DFS will both work for the connectivity analysis, and Dijkstra's will give you the shortest time to travel. I implemented Dijkstra using a linear scan rather than a priority queue. Assuming expected O(1) hash-map operations, its expected time complexity is $O(V^2 + E)$. Severe hash collisions can increase the worst-case running time. A priority queue-based implementation would be better for scaling up to larger networks.

In Module 2, I went with open addressing with linear probing rather than separate chaining (this avoids the need for any extra linked list structures and makes the collision resolution quite clear). The downside of linear probing is, of course, primary clustering, which can lead to a degradation of lookup performance. Automatic resizing and tombstone management were therefore implemented to maintain acceptable performance and to ensure correct search behaviour after a deletion has occurred.

Module 3 uses the array-based Max Heap to implement priority scheduling. The percolation step in either an insertion or an extraction takes at most O(log q), where q is the number of requests in the heap. Printing out the entire heap is O(q) work, and updating the archive has its own overhead (sorting and DB I/O as discussed in Section 5.2). The priority formula depends on estimated feedstock availability and transportation time. A key design issue was ensuring consistency between the modules. The scheduler re-validates the farm information prior to collection and will skip any request that is associated with a record that has been deleted or modified. This improves system reliability at the cost of additional validation and coordination.

Module 4 was all about the trade-offs between Merge Sort and Quick Sort. Merge Sort is stable and has the predictable O(n log n) performance, at the cost of requiring O(n) auxiliary space. Quick Sort's in-place implementation uses O(log n) auxiliary stack space. But the non-mutating version in the benchmark needs O(n) extra space for the copy of the input, though its worst-case time is still $O(n^2)$. The results of the experiment showed that neither algorithm was in any way consistently the faster of the two.

Overall, the system's real strength is the integration of the four manually implemented modules with input validation, testing, and the persistent collection of records. The weaknesses are the approximate nature of the transport times, the educational feedstock values, and the additional processing costs involved in the repeated sorting of the archived records. The project has served to underline the importance of balancing algorithmic efficiency, correctness, maintainability, and the practicalities of the system.

8. Limitations and Assumptions

The dataset was collected on 19 September 2026 by selecting Western Australia on the Farm Transparency Project map. The CSV that was submitted is a snapshot of 125 records and is not in any way a complete or up-to-date register. I did check the source URLs for all the records, but only spot-

checked five of the website listings, and those checks cannot in any way verify the status of every address or facility or any changes. Three of the FarmIDs have been restored from matching URLs, and the source address fields that were incomplete have been left as they were rather than attempted to be inferred.

The Pinjarra hub is purely hypothetical, with no assessment of land suitability, no permits, no facility capacity, and no commercial viability. The graph links are the selected educational links, not actual roads. The transport weights are derived from haversine distance, a 1.30 road distance factor, and 60km/h. Dijkstra therefore returns the shortest path in the modelled network, not the actual shortest driving route. The isolated farm is a test case and does not in any way indicate that the farm is physically inaccessible.

Estimated Feedstock values are purely educational request inputs. The priority scores don't represent actual waste availability. The scheduler will only take one pending request per farm and will check for any deleted or changed records prior to dispatch. It doesn't do any tour optimization at all (no multi-stop tours, no vehicle capacity, no loading times, no delivery windows, no operational cost). The sorting benchmarks re-use the routes from the 124 reachable farms to create the same historic request events over and over again rather than any new farm observations.

Finally, archiving does the sorting and persistence of the request history, which adds overhead on top of the heap operations. The benchmark timings were taken on one host with fixed seeds and do not include the time for dataset preparation or database I/O. They should not be taken as indicative of production performance.

9. Generative AI Declaration

I used generative AI tools as supplementary learning and research resources throughout the assignment. Their primary purpose was to help identify relevant academic references, summarise theoretical concepts, and strengthen my understanding of the data structures and algorithms covered in COMP5008.

In particular, Generative AI helped identify potential reference materials on graph traversal, Dijkstra's shortest-path algorithm, hash tables, binary heaps, Merge Sort, and Quick Sort. It also summarised complex theoretical explanations into more accessible concepts, helping me review algorithm behaviour, time and space complexity, and the advantages and limitations of different implementation approaches.

I also used Generative AI as a self-testing tool to check my understanding of the algorithms. I asked it conceptual questions, looked at the principles of the algorithms, and considered various scenarios to see where my understanding was lacking.

Finally, Generative AI supported the review of the assignment requirements by helping organize the assessment specifications into a checklist. This allowed me to systematically compare the required deliverables against my work and identify potentially overlooked requirements, test cases, documentation elements, and edge cases.

These activities supported independent learning, improved conceptual understanding, and enabled a more systematic review of the assignment requirements.

10. References

[1] “Farm Transparency Map: Interactive Map of Australian Factory Farms and Slaughterhouses - Farm Transparency Project,” www.farmtransparency.org. https://www.farmtransparency.org/map (accessed Sep. 19, 2026).

[2] E. W. Dijkstra, “A Note on Two Problems in Connexion With Graphs,” Numerische Mathematik, vol. 1, no. 1, pp. 269–271, Dec. 1959, doi: 10.1007/bf01386390.

[3] R. Sedgewick and K. Wayne, “MergeBU,” Algorithms 4/e, Aug. 25, 2026. https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/MergeBU.html (accessed Oct. 10, 2026).

[4] R. Sedgewick and K. Wayne, “QuickX (Algorithms 4/E),” in Princeton.Edu, 2026. Accessed: Oct. 10, 2026. [Online]. Available: https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/QuickX.html

[5] R. Sedgewick, K. Wayne, “Quick3way (Algorithms 4/E),” edu.princeton.cs.algs4, Aug. 25, 2026. https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/Quick3way.html (accessed Oct. 10, 2026).

[6] “Timeit — Measure Execution Time of Small Code Snippets,” Python documentation, 2025. https://docs.python.org/3.12/library/timeit.html (accessed Oct. 10, 2026).

Where to inspect primary experiment evidence

data/source_verification.csv records profile spot checks and exact ID evidence.

sample_output/test_results.txt contains the full successful test run.

sample_output/validation_summary.json records counts and graph paths.

sample_output/sorting/benchmark_metadata.json records input hashes, environment, untimed counters, and per-case validation.

sample_output/sorting/timing_samples.csv contains all 126 measured samples. Each timing result is tied to the supplied input and its two validated sorted CSV outputs.

Appendix A - Evidence and Heap Trace

This compact trace shows RequestIDs in actual array order, with the root at index 0. F, T, and P are listed in Section 6.3; full indexed priorities and swap details are preserved in sample_output/heap_traces.txt.

#	Operation	Heap array after the operation
1	INSERT C001	[C001]
2	INSERT C002	[C001, C002]
3	INSERT C003	[C001, C002, C003]
4	INSERT C004	[C001, C004, C003, C002]
5	INSERT C005	[C001, C004, C003, C002, C005]
6	INSERT C006	[C001, C004, C003, C002, C005, C006]
7	INSERT C007	[C001, C004, C003, C002, C005, C006, C007]
8	INSERT C008	[C001, C004, C003, C008, C005, C006, C007, C002]
9	INSERT C009	[C001, C004, C003, C008, C005, C006, C007, C002, C009]
10	INSERT C010	[C001, C004, C003, C008, C010, C006, C007, C002, C009, C005]
11	EXTRACT C001	[C004, C008, C003, C002, C010, C006, C007, C005, C009]
12	EXTRACT C004	[C003, C008, C006, C002, C010, C009, C007, C005]
13	EXTRACT C003	[C008, C002, C006, C005, C010, C009, C007]
14	EXTRACT C008	[C002, C007, C006, C005, C010, C009]
15	EXTRACT C002	[C006, C007, C009, C005, C010]
16	EXTRACT C006	[C007, C010, C009, C005]
17	EXTRACT C007	[C009, C010, C005]
18	EXTRACT C009	[C010, C005]
19	EXTRACT C010	[C005]
20	EXTRACT C005	[]
21	EXTRACT_EMPTY	[]

Graph/hash logs: test_network.txt, full_network.txt, hash_demo.txt, and hash_load_factors.csv. Integrated logs/JSON/CSV/SQLite: sample_output/integration/ and its deleted_farm/ scenario. Sorting: data/sort_inputs/ and sample_output/sorting/. UML source/renderings: docs/module_integration_uml.puml, .svg, and .png. README.md gives the folder map and all run commands.